

write

קריאת המערכת `write()` היא הפעולה שבאמצעותה תוכנית במרחב המשתמש (User Space) מעבירה נתונים ומכתבת אותם אל קובץ פתוח. במסגרת עבודה מול **Linux Kernel 6.14**, קריאה זו מנוהלת לרוב באמצעות מנגנון **Buffered I/O (קלט/פלט ממוקד מטמון)**, שבו הנתונים נכתבים תחילה אל זיכרון ה-RAM (ה-Page Cache) ורק בהמשך מסונכרנים אל התקן ה-SSD הפיזי, אלא אם כן הוגדרו דגלי סנכרון מיידי.

להלן האלגוריתם המלא והמפורט של קריאת המערכת `write()`, מתחילת הפנייה ועד לניחול הנתונים בקרנל.

אלגוריתם ה-`write()` – שלב אחר שלב (מבט על)

1. **שלב כניסה לקרנל ואיתור אובייקט הקובץ** (`fdget / sys_write`): מעבר מ-Ring 3 ל-Ring 0 ומיפוי ה-FD אל אובייקט הקובץ הפתוח בזיכרון (`struct file`).
2. **שלב בדיקת תקינות והרשאות כתיבה** (`f_mode`): אימות שהקובץ אכן נפתח במקור במצב המאפשר כתיבה.
3. **שלב ניהול מצבי כתיבה מיוחדים ונעילות** (`Locks & O_APPEND`): בדיקת דגלי הוספה לסוף הקובץ ותיאום נעילות מקבילות.
4. **שלב הפעלת ממשק ה-VFS ומערכת הקבצים** (`ext4_file_write_iter / vfs_write`): ניתוב הבקשה דרך שכבת ההפשטה למתודות הייעודיות של `ext4`.
5. **שלב הבאה/הקצאה של דפי זיכרון בטוחים** (`Read- & Page Cache Allocation`): **Modify-Write**: איתור או טעינה של דף הזיכרון הרלוונטי (KB4) ב-RAM, תוך טיפול בכתיבה חלקית (`Partial Block`).
6. **שלב העתקת הנתונים מרחב המשתמש לקרנל** (`copy_from_user`): העברה בטוחה של הנתונים מבאפר התהליך אל תוך ה-Page Cache וסימון הדף כ-"מלוכלך" (`SetPageDirty`).
7. **שלב עדכון המטא-דאטה, המיקום וה-Inode** (`i_size & f_pos`): קידומת מצביע הכתיבה (`f_pos`), עדכון גודל הקובץ ב-In-Core Inode, וסימון ה-Inode כמלוכלך.
8. **שלב הקצאת בלוקים פיזיים** (`Extents & Block Allocation`): במקרה שהכתיבה מרחיבה את הקובץ אל מעבר לבלוקים קיימים, מערכת הקבצים מקצה בלוקים חדשים (או משתמשת בהקצאה דחויה - `Delayed Allocation`).
9. **שלב סיכזור ורישום ביומן** (`Journaling / Writeback`): רישום השינויים ביומן המערכת (ב-jbd2 ל-ext4) והכנת הנתונים להעברה פיזית ל-SSD על ידי תהליכי הרקע של הקרנל.
10. **שלב סיום והחזרה למרחב המשתמש**: חזרה ל-Ring 3 והחזרת מספר הבייטים שנכתבו בפועל.

הסבר מלא ומפורט לכל צעד באלגוריתם

- שלב 1: כניסה לקרנל ואיתור אובייקט הקובץ (fdget / sys_write)
 - מה קורה בפועל: תוכנית במרחב המשתמש קוראת לפונקציה:

ssize_t write(int fd, const void *buf, size_t count);
והבקשה מגיעה ל-Ring 0 ל-Ring 3 מעבירה את המעבד מ-syscall לפקודת האסמבלי של ה-
ksys_write (או sys_write לפונקציית הליבה).

- איתור ה-File Descriptor: הקרנל שולף את ה-fd המבוקש מטבלת תיאורי הקבצים של התהליך (files-<current) כדי לאתר את המצביע אל אובייקט הקישור struct file.*
אם ה-fd אינו תקין, הקרנל מחזיר שגיאת EBADF.

שלב 2: בדיקת תקינות והרשאות כתיבה (f_mode)

- מה קורה בפועל: הקרנל בודק את שדה ההרשאות באובייקט ה-struct file (שדה f_mode). הוא מוודא שהקובץ נפתח במקור עם דגל כתיבה חוקי (כגון O_WRONLY או O_RDWR).
- אם הקובץ נפתח אך ורק לקריאה (למשל עם O_RDONLY), הקרנל חוסם את הפעולה מיד ומחזיר שגיאת EBADF.

שלב 3: ניהול מצבי כתיבה מיוחדים ונעילות (Locks & O_APPEND)

- מה קורה בפועל: הקרנל בודק האם הקובץ נפתח עם דגל ההוספה O_APPEND.
 - אם כן, לפני כל כתיבה, הקרנל מעדכן אוטומטית את מצביע המיקום של הקובץ (f_pos) להצביע בדיוק על סופו הנוכחי של הקובץ (ערך ה-i_size של ה-Inode), כדי להבטיח שהנתונים החדשים יתווספו תמיד לסוף הקובץ מבלי לדרוס מידע קיים (קריטי בסביבת מרובה-תהליכים).
 - במקביל, הקרנל מפעיל מנעולים (Locks) ברמת ה-In-Core Inode (כגון i_rwsem) כדי למנוע מצבי תחרות (Race Conditions) שבהם שני תהליכים מנסים לכתוב לאותו קובץ בדיוק באותו מיקרו-שניה.

שלב 4: הפעלת ממשק ה-VFS ומערכת הקבצים (ext4_file_write_iter / vfs_write)

- מה קורה בפועל: הקרנל מפעיל את פונקציית התווך הגנרית של ה-VFS בשם vfs_write, המנתקת את הבקשה ומעבירה אותה למערכת הקבצים הספציפית דרך מתודת ה-write_iter (לדוגמה, ext4_file_write_iter). מתודה זו מנהלת את אופן כתיבת הנתונים לעץ ה-Extents של ext4.

שלב 5: הבאה/הקצאה של דפי זיכרון בטוחים (Read-Modify- & Page Cache Allocation) (Write)

זהו שלב הלב הפועם של כתיבה ממוקמת מטמון (Buffered Write):

- הקרנל מחשב איזה עמוד זיכרון ב-RAM (Page, בגודל KB4) תואם למיקום הכתיבה הנוכחי (f_pos).
- הוא בודק במטמון האם עמוד זה כבר קיים בזיכרון.

- אם העמוד אינו קיים בזיכרון (Cache Miss) והכתיבה מיועדת לעדכן חלק בלבד ממנו (Partial Block Write) – למשל כתיבה של 100 בייט באמצע בלוק של 4KB, הקרנל נאלץ לבצע פעולת **Read-Modify-Write**: הוא קורא תחילה את הבלוק הפיזי מה-SSD אל תוך ה-RAM, כדי לוודא ששאר החלקים של הבלוק לא יושחתו.

- אם מדובר בכתיבה מלאה של עמוד שלם (4KB), הקרנל פשוט מקצה עמוד זיכרון ריק לחלוטין ב-RAM מבלי לקרוא מהדיסק.

שלב 6: העתקת הנתונים ממרחב המשתמש לקרנל (copy_from_user)

- **מה קורה בפועל:** הקרנל מפעיל את פונקציית ההעברה המוגנת `copy_from_user`. פונקציה זו מעתיקה בבטחה את כמות הבייטים המבוקשת מתוך באפר התהליך שבמרחב המשתמש אל תוך דפי הזיכרון של ה-Page Cache בזיכרון הליבה.
- **סימון הדף כמלוכלך (SetPageDirty):** ברגע שהנתונים נכתבו בהצלחה לזיכרון ה-RAM, הקרנל מסמן את דף הזיכרון הזה בדגל **Dirty Page**. סימון זה מעיד שהנתונים בזיכרון שונים כעת ממה שקיים על גבי הדיסק הפיזי, ושהם דורשים סינכרון עתידי.

שלב 7: עדכון המטא-דאטה, המיקום וה-Inode (i_size & f_pos) מה קורה בפועל:

1. הקרנל מקדם את מצביע המיקום של הקובץ (**File Offset**, השמור בשדה `f_pos`) קדימה בדיוק במספר הבייטים שנכתבו.
2. אם הכתיבה הרחיבה את הקובץ מעבר לגודלו הקודם, הקרנל מעדכן את שדה גודל הקובץ (`i_size`) בתוך ה-Inode ה-In-Core.
3. הקרנל מסמן גם את ה-Inode ה-In-Core עצמו כדגל מלוכלך (`I_DIRTY_DATASYNC`), כדי שעדכוני המטא-דאטה יירשמו במערכת.

שלב 8: הקצאת בלוקים פיזיים (Extents & Block Allocation)

- **מה קורה בפועל:** אם הכתיבה דרשה הקצאת מקום חדשה על הדיסק (מפני שהקובץ גדל מעבר לבלוקים שהוקצו לו בעבר):

- ב-ext4 נעשה לרוב שימוש במנגנון **Delayed Allocation** (**הקצאה דחוייה**): הקרנל דוחה את ההקצאה הפיזית של בלוקים על ה-SSD כדי לייעל אותה ולמנוע פיצול (Fragmentation), תוך שהוא שומר את הנתונים בבטחה ב-Page Cache.
- כאשר ההקצאה מתבצעת לבסוף, מנגנון ה-Extents של ext4 מעדכן את עץ המיפוי של הקובץ ב-Inode, ומצביע על מתחמי הבלוקים החדשים שיוקצו על ה-SSD.

שלב 9: סינכרון ורישום ביומן (Journaling / Writeback)

- **מה קורה בפועל:**
- כדי להבטיח את אמינות מערכת הקבצים במקרה של קריסת מערכת, מערכת הקבצים ext4 רושמת את שינויי המטא-דאטה ליומן הטרנזקציות (**Journaling**) המנוהל על ידי תת-המערכת `jbd2`.

- הנתונים עצמם שוהים כרגע בטוחים ב-Page Cache. הם יועברו וייכתבו פיזית ל-SSD על ידי תהליכי רקע של הקרנל (כגון חוטי ה-**Writeback** / flusher threads) או באופן מיידי אם התהליך פתח את הקובץ עם דגלי סינכרון מחמירים (O_SYNC או O_DSYNC).

שלב 10: סיום והחזרה למרחב המשתמש

- **מה קורה בפועל:** הקרנל משחרר את הנעילות מעל ה-Inode, מעביר את המעבד בחזרה מ-Ring 0 ל-Ring 3, ומחזיר את מספר הבייטים שנכתבו בפועל (מספר שלם חיובי התואם ל-count) אל פונקציית ה-write() במרחב המשתמש.